Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	XXXXXX
Project Name	Internal Job Mobility Assistant
Maximum Marks	5 Marks

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples
1	Folder/Directory Structure	The project follows a modular folder structure separating frontend, backend, database models, API routes, and AI services to improve maintainability and scalability.	Separate folders such as frontend/, backend/, components/, models/, routes/, services/, and utils/.
2	Naming Conventions	Consistent naming conventions were followed throughout the project. Variables and functions use camelCase, React components use PascalCase, and Python files use snake_case.	Examples: LoginRegister.tsx, career_path_service.py, calculateJobMatch(), user_profile.
3	Code Readability & Comments	Meaningful variable names, inline comments, and function-level documentation were used to improve readability and ease future maintenance.	Comments added in ai_service.py, API endpoint descriptions in FastAPI, and descriptive variable names such as compatibility_score.
4	Modularity & Reusability	Code was divided into reusable components and services to avoid duplication and promote maintainability.	Reusable React components, separate authentication service, AI service module, and database utility functions reused across multiple pages.
5	Formatting & Linting Tools	Standard code formatting practices and IDE formatting tools were used to maintain consistency across the codebase.	Visual Studio Code auto-formatting, PEP-8 guidelines for Python, consistent indentation and code styling practices.

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples
6	Error Handling Patterns	Exception handling and validation mechanisms were implemented to ensure robust application behavior and graceful error responses.	try-except blocks in FastAPI APIs, input validation using Pydantic models, centralized error responses for invalid requests.